

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**

**Ордена трудового Красного Знамени федеральное государственное
бюджетное**

образовательное учреждение высшего образования

«Московский технический университет связи и информатики»

Кафедра Математическая кибернетика и информационные технологии

Отчет по лабораторной работе №4.

“DI Разработка веб-приложения на Spring Boot. Работа с данными”

Выполнил: студент группы БВТ2402

Скопа Михаил Вячеславович

Москва, 2026

Цель работы: Научиться работать с данными в Spring Boot-приложении с использованием Spring Data JPA и Hibernate: подключать приложение к базе данных PostgreSQL, создавать JPA-сущности и репозитории, реализовывать базовые CRUD-операции, настраивать связи между таблицами, использовать транзакции, валидировать входные данные и обрабатывать ошибки.

Задание для выполнения лабораторной работы

1. Вынесите повторяющийся код преобразования Notification в NotificationDto в отдельный метод mapToDto() или в отдельный mapper-класс.

```
private NotificationDto mapToDto(Notification notification) { 7 usages
    return NotificationDto.builder()
        .id(notification.getId())
        .title(notification.getTitle())
        .message(notification.getMessage())
        .recipientId(notification.getRecipientId())
        .status(notification.getStatus())
        .channel(notification.getChannel())
        .createdAt(notification.getCreatedAt())
        .build();
}
```

Вынес повторяющийся код в отдельный метод в NotificationController.

```
public class NotificationController {
    private final NotificationService notificationService;

    @PostMapping("/add")
    public NotificationDto createNotification(@RequestBody @Valid NotificationDto request) {
        Notification response = notificationService.createNotification(request);
        return mapToDto(response);
    }

    @GetMapping("/all")
    public List<NotificationDto> getAllNotifications() {
        return notificationService.getAllNotifications().stream()
            .map(this::mapToDto)
            .collect(Collectors.toList());
    }

    @GetMapping("/{id}")
    public NotificationDto getNotificationById(@PathVariable Long id) {
        Notification notification = notificationService.getNotificationById(id);
        return mapToDto(notification);
    }

    @PutMapping("/{id}")
    public NotificationDto updateNotification(@PathVariable Long id, @RequestBody @Valid NotificationDto request) {
        Notification response = notificationService.updateNotification(id, request);
        return mapToDto(response);
    }

    @DeleteMapping("/{id}")
    public String deleteNotification(@PathVariable Long id) {
        notificationService.deleteNotification(id);
        return "Уведомление удалено";
    }
}
```

2. Реализуйте проверку номера телефона с помощью @Pattern.

В userDto добавил блок проверки номера телефона.

```
@NotBlank(message = "Телефон не может быть пустым")
@Pattern(
    regexp = "^((\\+7|8)[0-9]{10})$",
    message = "Номер телефона должен начинаться с +7 или 8 и содержать 11 цифр"
)
```

3. Добавьте метод репозитория поиска уведомлений по статусу и каналу одновременно.

В NotificationRepository добавил:

```
List<Notification> findByStatusAndChannel(NotificationStatus status, NotificationChannel channel);
```

В NotificationService добавил:

```
public List<Notification> getByStatusAndChannel(NotificationStatus status, NotificationChannel channel) {
    return notificationRepository.findByStatusAndChannel(status, channel);
}
```

В NotificationController добавил:

```
@GetMapping("/filter")
public List<NotificationDto> getByStatusAndChannel(
    @RequestParam NotificationStatus status,
    @RequestParam NotificationChannel channel) {
    return notificationService.getByStatusAndChannel(status, channel).stream()
        .map(this::mapToDto)
        .toList();
}
```

GET

http://localhost:8080/notifications/filter?status=PENDING&channel=EMAIL

Docs Params Authorization Headers 8 Body Scripts Tests Settings

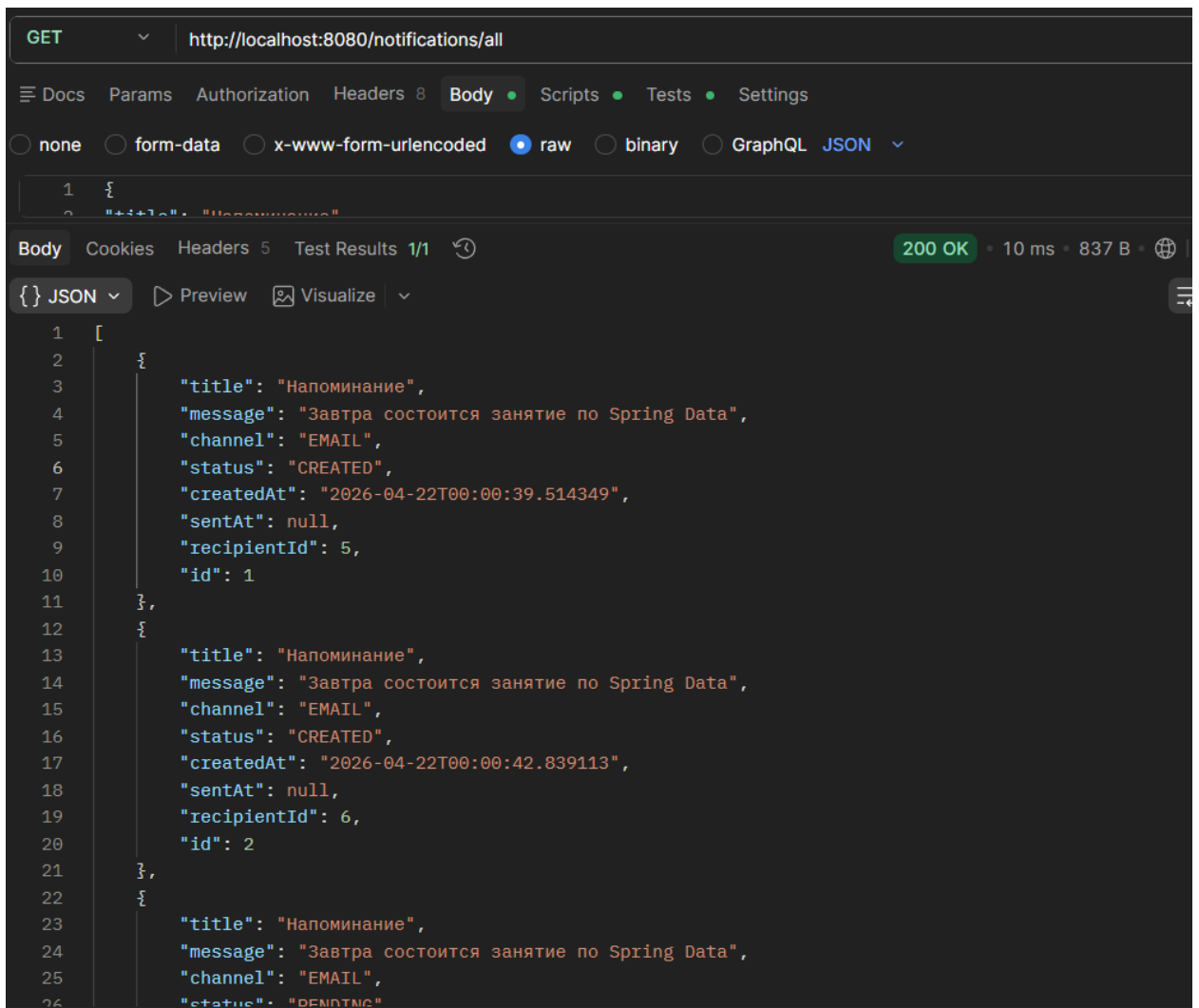
☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
1 {
2   "title": "Напоминание",
3   "message": "Завтра состоится занятие по Spring Data",
4   "channel": "EMAIL",
5   "recipientId": 5
6 }
```

Body Cookies Headers 5 Test Results 1/1 200 OK

{ } JSON Preview Visualize

```
1 [
2   {
3     "title": "Напоминание",
4     "message": "Завтра состоится занятие по Spring Data",
5     "channel": "EMAIL",
6     "status": "PENDING",
7     "createdAt": "2026-04-22T01:38:37.688224",
8     "sentAt": null,
9     "recipientId": 5,
10    "id": 11
11  }
12 ]
```



Фильтр работает.

- Добавьте метод репозитория с сортировкой уведомлений по дате создания в порядке убывания.

В NotificationController добавил:

```
@GetMapping("/sorted")
public List<NotificationDto> getAllSorted() {
    return notificationService.getAllNotificationsSorted().stream()
        .map(this::mapToDto)
        .toList();
}
```

В NotificationRepository добавил:

```
List<Notification> findAllByOrderByCreatedAtDesc(); no usages
```

В NotificationService добавил:

```
public List<Notification> getAllNotificationsSorted() {
    return notificationRepository.findAllByOrderByCreatedAtDesc();
}
```

Уведомления сортируются по дате создания в порядке убывания.

```
[
  {
    "title": "Напоминание",
    "message": "Завтра состоится занятие по Spring Data",
    "channel": "EMAIL",
    "status": "PENDING",
    "createdAt": "2026-04-22T01:44:16.214686",
    "sentAt": null,
    "recipientId": 5,
    "id": 13
  },
  {
    "title": "Напоминание",
    "message": "Завтра состоится занятие по Spring Data",
    "channel": "EMAIL",
    "status": "PENDING",
    "createdAt": "2026-04-22T01:44:15.845734",
    "sentAt": null,
    "recipientId": 5,
    "id": 12
  },
  {
    "title": "Напоминание",
    "message": "Завтра состоится занятие по Spring Data",
    "channel": "EMAIL",
    "status": "PENDING",
    "createdAt": "2026-04-22T01:44:15.845734",
    "sentAt": null,
    "recipientId": 5,
    "id": 11
  }
]
```

5. Реализуйте собственный @Query для поиска уведомлений пользователя по его recipientId и статусу.

В NotificationRepository добавил:

```
@Query("SELECT n FROM Notification n WHERE n.recipientId = :recipientId AND n.status = :status")
List<Notification> findByRecipientAndStatus(
    @Param("recipientId") Long recipientId,
    @Param("status") NotificationStatus status
);
```

В NotificationService добавил:

```
public List<Notification> getByRecipientAndStatus(Long recipientId, NotificationStatus status) {
    return notificationRepository.findByRecipientAndStatus(recipientId, status);
}
```

В NotificationController добавил:

```
@GetMapping("/search")
public List<NotificationDto> searchNotifications(
    @RequestParam Long recipientId,
    @RequestParam NotificationStatus status) {
    return notificationService.getByRecipientAndStatus(recipientId, status).stream()
        .map(this::mapToDto)
        .toList();
}
```

GET ▼ http://localhost:8080/notifications/search?recipientId=5&status=PENDING

Docs Params Authorization Headers 8 Body Scripts Tests Settings

Query Params

Key	Value	Description
✓ recipientId	5	

Body Cookies Headers 5 Test Results 1/1 🕒 200 OK • 9 ms

{ } JSON ▼ Preview Visualize ▼

```
1  [
2    {
3      "title": "Напоминание",
4      "message": "Завтра состоится занятие по Spring Data",
5      "channel": "EMAIL",
6      "status": "PENDING",
7      "createdAt": "2026-04-22T01:38:37.688224",
8      "sentAt": null,
9      "recipientId": 5,
10     "id": 11
11   },
```

6. Добавьте в NotificationService метод, который при установке статуса SENT автоматически записывает значение в поле sentAt.

В NotificationService добавил:

```
public Notification updateNotificationStatus(Long id, NotificationStatus newStatus) { no usages
    Notification notification = notificationRepository.findById(id)
        .orElseThrow(() -> new RuntimeException("Уведомление не найдено"));

    notification.setStatus(newStatus);

    if (newStatus == NotificationStatus.SENT) {
        notification.setSentAt(LocalDate.now());
    }

    return notificationRepository.save(notification);
}
```

```
1  {
2    "title": "Новое уведомление",
3    "message": "Привет! Это текст сообщения для проверки 400 ошибки.",
4    "recipientId": 6,
5    "status": "SENT",
6    "channel": "EMAIL",
7    "phoneNumber": "+79991234567"
8  }

dy Cookies Headers 5 Test Results 1/1 200 OK
JSON Preview Visualize
1  {
2    "title": "Новое уведомление",
3    "message": "Привет! Это текст сообщения для проверки 400 ошибки.",
4    "channel": "EMAIL",
5    "status": "SENT",
6    "createdAt": "2026-04-22T02:15:02.129669",
7    "sentAt": "2026-04-22T02:15:02.1118983",
8    "recipientId": 6,
9    "id": 27
10 }
```

7. Проверьте поведение приложения при попытке создать уведомление для несуществующего пользователя.

При попытке создать уведомление для пользователя с несуществующим ID (999999), база данных заблокировала операцию. Это произошло благодаря ограничению Foreign Key на поле `recipient_id`. Система выдала исключение `DataIntegrityViolationException`, что подтверждает надежность связей между таблицами и невозможность появления "свободных" записей

Вывод: Научился работать с данными в Spring Boot-приложении с использованием Spring Data JPA и Hibernate: подключать приложение к базе данных PostgreSQL, создавать JPA-сущности и репозитории, реализовывать базовые CRUD-операции, настраивать связи между таблицами, использовать транзакции, валидировать входные данные и обрабатывать ошибки.